Multi-Class Text Classification of News Headlines: A Comparative Evaluation of Preprocessing, TF-IDF, Skip-gram Word2Vec, and Neural Sequence Models

Tanjum Ibnul Mahmud Prithu
Department of Bachelor of Science in Computer Science (CS)
Dhaka, Bangladesh
Email: tanjum.ibnul.mahmud@g.bracu.ac.bd
Student ID: 22241108

Tahmid Mahdi
Department of Computer Science and Engineering (CSE)
Dhaka, Bangladesh
Email: tahmid.mahdi@g.bracu.ac.bd
Student ID: 22299400

***Abstract*—This report presents a complete experimental study for multi-class news headline classification. The task is to assign each headline to one of four categories: Business, Science and Technology, Sports, and World News. The work compares three preprocessing conditions, namely no preprocessing, extreme preprocessing, and optimum preprocessing, together with TF-IDF and Skip-gram Word2Vec representations. Thirty validation runs were conducted using Logistic Regression, a feed-forward Deep Neural Network, SimpleRNN, GRU, LSTM, and bidirectional recurrent variants. The best validation configuration for each model and preprocessing setting was then re-trained and evaluated on a held-out test set of 12,000 headlines. The strongest final model was Logistic Regression with TF-IDF and optimum preprocessing, achieving 0.915833 test accuracy and 0.915482 macro F1-score. The weakest final model was a SimpleRNN using Skip-gram Word2Vec with extreme preprocessing, which collapsed to 0.250167 accuracy and 0.100346 macro F1-score. The findings show that, for short news text, a carefully tuned sparse representation can outperform heavier neural sequence models when the preprocessing pipeline preserves useful lexical information.**

***Index Terms*—Natural language processing, multi-class classification, news headlines, TF-IDF, Word2Vec, Logistic Regression, Deep Neural Network, RNN, GRU, LSTM**

I. Introduction

Automatic text classification is one of the most widely used applications of natural language processing (NLP). It is used in news recommendation, search engines, spam filtering, digital libraries, content moderation, and large-scale information retrieval. In this project, the classification task is focused on news headlines. Although headlines are shorter than full news articles, they often contain strong topic cues, named entities, event terms, and domain-specific keywords. At the same time, their brevity makes the task sensitive to preprocessing and feature representation. A useful word removed during preprocessing, or a noisy token retained during vectorisation, can directly change the decision boundary of the classifier.

The goal of this project is to design and evaluate a full NLP pipeline for multi-class classification. The dataset contains English news headlines and four labels: *Business*, *Science and Technology*, *Sports*, and *World News*. The notebook attached with this project implements data loading, exploratory data analysis (EDA), text cleaning, multiple preprocessing pipelines, vectorisation and embedding methods, 30 validation experiments, final test evaluation, confusion matrices, result tables, and comparative plots.

This report follows the IEEE conference format and presents the project as a publishable experimental study. The main questions investigated are as follows. First, does heavy preprocessing improve or reduce classification performance? Second, does a classical TF-IDF representation remain competitive against neural word embeddings for short text classification? Third, which model architecture provides the strongest balance between accuracy, macro F1-score, and runtime? Fourth, which configuration should be selected as the final system based on objective evaluation rather than intuition alone?

The findings are clear. TF-IDF with Logistic Regression is not only highly competitive but is the best-performing final method in the experiments. Optimum preprocessing performs better than extreme preprocessing because it removes technical noise while preserving important lexical and semantic signals. Deep recurrent models can perform well, especially GRU and LSTM variants, but they are more sensitive to preprocessing and model stability. The poor results of some SimpleRNN configurations show that sequence models are not automatically superior unless the representation and architecture are appropriate for the dataset.

II. Methodology

A. Dataset Description

The project uses two comma-separated value files: one training file and one testing file. Each row contains a news headline and its corresponding news topic. The raw training set contains 97,851 rows and two columns. The raw test set contains 12,000 rows and two columns. No missing values were found in either split. However, the training set contained 20,105 duplicate rows. These duplicate training examples were removed before modelling, reducing the training set from 97,851 to 77,746 rows. The test set remained unchanged at 12,000 rows because it had no duplicate rows.

TABLE I
DATASET OVERVIEW AFTER INITIAL INSPECTION

Property	Training Set	Testing Set
Raw rows	97,851	12,000
Columns	2	2
Missing values	0	0
Duplicate rows	20,105	0
Rows after cleanup	77,746	12,000
Topic classes	4	4

TABLE II
CLASS DISTRIBUTION IN TRAINING AND TESTING DATA

News Topic	Train Count	Test Count
Business	13,625	3,000
Science and Technology	25,972	3,000
Sports	41,268	3,000
World News	16,986	3,000

Table II lists the class counts. The training set is imbalanced: Sports has the highest count, while Business has the lowest count. In contrast, the testing set is perfectly balanced with 3,000 examples per class. This mismatch makes macro F1-score important because macro F1 gives equal importance to every class, regardless of how frequently it appears in the training data.

B. Exploratory Data Analysis

The EDA stage was designed to understand the structure, noise, balance, length, and vocabulary of the data before deciding on preprocessing. Raw examples contained HTML tags, repeated phrases such as "News Headlines", line breaks, and publication-style wording. Therefore, a light EDA-only cleaning function was first applied to make the text readable for analysis. This cleaning removed HTML tags, non-alphabetic characters, repeated spaces, and casing differences, but it was not used as the final modelling preprocessing by itself.

Fig. 1 and Fig. 2 show the class distribution of the training and testing data. The training distribution is skewed toward Sports, followed by Science and Technology, World News, and Business. The test distribution is balanced. This is useful for fair testing because each class contributes equally to the final accuracy and macro F1-score. However, it also means the model must generalise from an imbalanced training distribution to a balanced evaluation setting.

Headline length was analysed using character length and word count. Table III summarises the descriptive statistics. The mean word count is about 39.57 words, and the median is 39 words. The maximum word count is 177 words, which confirms that some rows include more than a short headline because of boilerplate news text and HTML formatting. Fig. 3 shows the full word count distribution. Most entries are concentrated around the middle range, but a right tail exists because of longer headline blocks. Fig. 4 shows word count by topic. World News has the highest average word count, while Sports is slightly shorter on average.

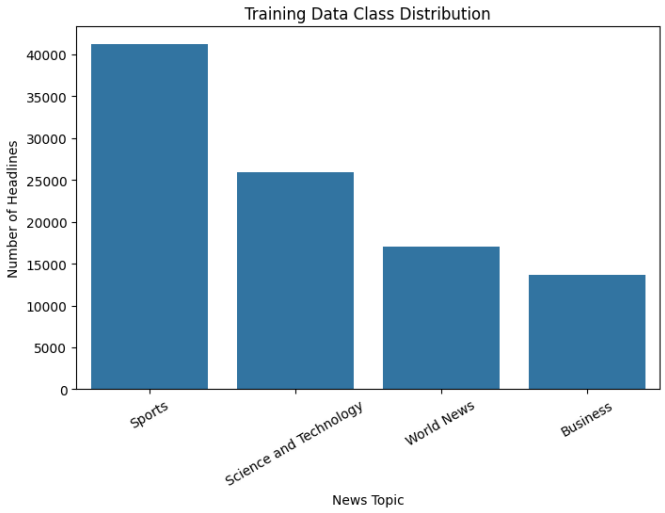

Fig. 1. Training data class distribution. Sports is the dominant class, while Business is the minority class.

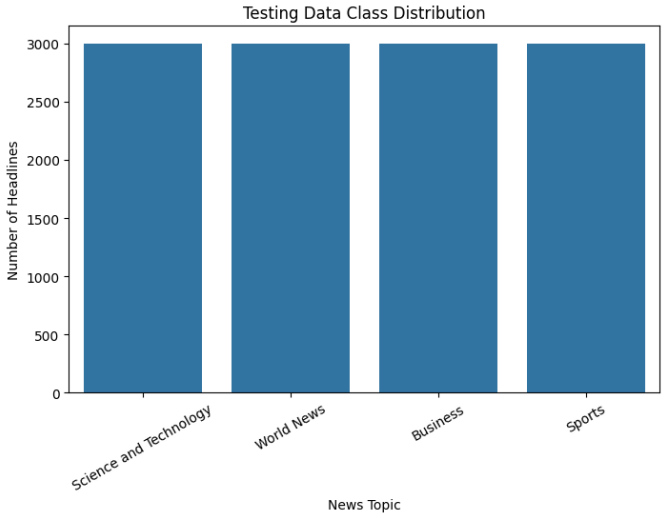

Fig. 2. Testing data class distribution. The final test set is balanced with 3,000 headlines per class.

TABLE III
CHARACTER LENGTH AND WORD COUNT STATISTICS FROM EDA

Statistic	Char Length	Word Count
mean	235.74	39.57
std	57.85	10.11
min	59.00	10.00
25%	200.00	33.00
50%	232.00	39.00
75%	264.00	45.00
max	970.00	177.00

Vocabulary analysis was performed after removing EDA stopwords and obvious HTML noise. Table V lists the top 15 frequent tokens from the output log, while Fig. 5 visualises the top 30 words. Words such as "game", "season", "team", and "win" provide clear Sports signals. Words such as "company",

TABLE IV
WORD COUNT BY NEWS TOPIC

Topic	Mean	Median	Min	Max
Business	39.39	39.00	11	127
Science and Technology	39.47	38.00	10	177
Sports	39.18	38.00	16	151
World News	40.83	41.00	15	150

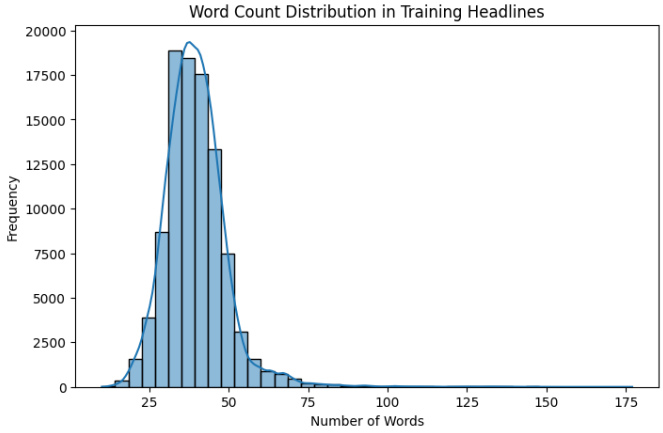

Fig. 3. Word count distribution in the training headlines after EDA cleaning.

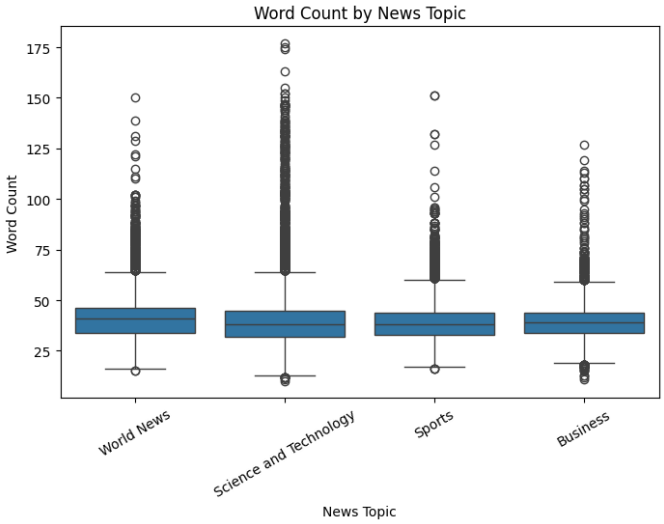

Fig. 4. Word count by news topic. World News has a slightly higher central tendency than the other classes.

"microsoft", "oil", and "prices" are more related to Business and Science and Technology. General corpus terms such as "said", "reuters", and weekday names occur frequently across classes and are therefore less class-specific.

The word cloud in Fig. 6 confirms that the dataset has strong recurring lexical cues. The most visible words are not random; they reflect news style, entities, and topic-specific vocabulary. This observation supports the use of TF-IDF because sparse lexical features can capture discriminative terms effectively for short text.

TABLE V
TOP FREQUENT WORDS IN TRAINING HEADLINES

Rank	Word	Frequency
1	new	16,908
2	said	13,016
3	reuters	11,347
4	first	9,584
5	two	8,632
6	year	7,961
7	world	7,743
8	one	7,638
9	game	7,025
10	last	6,214
11	not	6,141
12	monday	5,734
13	wednesday	5,653
14	thursday	5,466
15	three	5,461

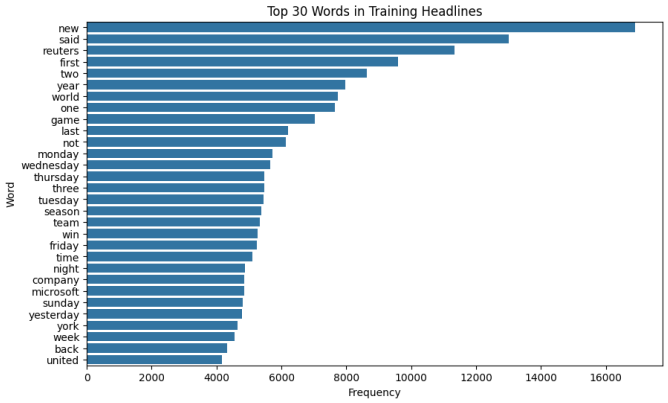

Fig. 5. Top 30 words in the training headlines after EDA cleaning and noise filtering.

Fig. 6. Word cloud for all training headlines. Larger words appear more frequently in the cleaned training vocabulary.

N-gram analysis was also conducted to identify phrase-level signals. Table VI shows the most frequent bigrams and trigrams. Expressions such as "new york", "united states", "prime minister", "oil prices", "boston red sox", and "international space station" carry topic information that individual words alone may not fully represent. This supports the inclusion of bigram TF-IDF configurations in the validation grid.

TABLE VI
TOP N-GRAM PATTERNS FROM EDA

Rank	Bigram	Freq.
1	new york	4,634
2	reuters reuters	2,853
3	red sox	1,999
4	united states	1,914
5	sports network	1,680
6	last night	1,383
7	prime minister	1,373
8	york reuters	1,268
9	oil prices	1,264
10	san francisco	1,255

Rank	Trigram	Freq.
1	new york reuters	1,268
2	boston red sox	669
3	new york yankees	525
4	quote profile research	484
5	major league baseball	347
6	canadian press canadian	264
7	press canadian press	264
8	san francisco giants	260

C. Preprocessing Pipelines

Three preprocessing pipelines were implemented to compare the effect of no cleaning, heavy linguistic reduction, and balanced cleaning.

1) No Preprocessing: The no preprocessing condition returns the raw string exactly as given. This means HTML tags, punctuation, casing, line breaks, and boilerplate phrases are retained. This baseline is useful because modern vectorisers and classifiers can sometimes learn to ignore frequent noise without manual cleaning.

2) Extreme Preprocessing: Extreme preprocessing applies HTML unescaping, HTML tag removal, URL removal, non-alphabetic character removal, whitespace normalisation, lowercasing, tokenisation, full English stopword removal, custom news-noise removal, WordNet lemmatisation, and Porter stemming. It can be expressed as a sequence of transformations:

$$X_{EP} = S\left(L\left(R\left(N(X)\right)\right)\right), \tag{1}$$

where N is basic normalisation, R removes stopwords and noise tokens, L lemmatises tokens, and S stems them. This pipeline reduces vocabulary size but can damage meaning. For example, "architecture" becomes "architectur", and inflected words are reduced into stemmed forms that may be less readable.

3) Optimum Preprocessing: Optimum preprocessing uses the same basic normalisation and lemmatisation, but it keeps negation words and removes stemming. Therefore, it is less destructive than extreme preprocessing:

$$X_{OP} = L\left(R_{keep\_neg}\left(N(X)\right)\right). \tag{2}$$

This pipeline is motivated by the EDA finding that the data contains meaningful short phrases. Removing too much context may reduce the ability of TF-IDF and neural models to recognise important topic patterns. Table VII shows examples from the notebook output. The optimum version keeps cleaner word forms, while the extreme version compresses words through stemming.

D. Word Representation Techniques

1) TF-IDF Representation: Term Frequency-Inverse Document Frequency (TF-IDF) represents each document as a sparse vector that gives higher weight to terms that are frequent in a document but less common across the corpus [1]. For a term t, document d, and corpus D, the TF-IDF weight is computed as:

$$\mathrm{tfidf}(t,d,D) = \mathrm{tf}(t,d) \times \left[\log\left(\frac{1+N}{1+\mathrm{df}(t)}\right)+1\right], \tag{3}$$

where N is the number of documents and $\mathrm{df}(t)$ is the number of documents containing t. The project explored vocabulary sizes of 8,000, 10,000, 20,000, and 30,000 features, unigram and unigram-bigram ranges, minimum document frequency of 2, and maximum document frequency of 0.95. These settings reduce extremely rare words and very common non-discriminative terms.

2) Skip-gram Word2Vec: Skip-gram Word2Vec learns dense word embeddings by predicting context words from a centre word [2]. The objective maximises the probability of neighbouring words around each target word:

$$\frac{1}{T}\sum_{t=1}^{T}\sum_{-c\le j\le c, j\neq 0}\log p(w_{t+j}|w_t), \tag{4}$$

where c is the context window size. In this project, Word2Vec was trained only on the training portion of each split. Embedding dimensions of 100 and 150 were explored, with a window size of 5, minimum token count of 2, and five Word2Vec epochs. The learned embedding matrix was used to initialise the embedding layer of the recurrent models.

E. Model Architectures

1) Logistic Regression: Logistic Regression is a strong classical baseline for text classification, especially when paired with TF-IDF. For an input vector x, class k, weights w_k, and bias b_k, the predicted probability is:

$$P(y=k|x) = \frac{\exp(w_k^\top x + b_k)}{\sum_{j=1}^{K}\exp(w_j^\top x + b_j)}. \tag{5}$$

The classifier selects the class with the largest probability. The model was trained using the L-BFGS solver from scikit-learn [3], with regularisation parameter C values of 1.0 and 2.0.

TABLE VII
EXAMPLES OF THE THREE PREPROCESSING OUTPUTS

Topic	Raw Headline Snippet	Extreme Preprocessing	Optimum Preprocessing
World News	¡html¿ ¡body¿ News Headlines:\n ¡br¿ ¡b¿ Londo.....	london erot gherkin win top architectur prize	london erotic gherkin win top architecture pri.....
Science and Technology	¡html¿ ¡body¿ News Headlines:\n ¡br¿ ¡b¿ Ex-Ne.....	netscreen ceo join start former ceo netscreen	netscreen ceo join start former ceo netscreen .....
Sports	¡html¿ ¡body¿ News Headlines:\n ¡br¿ ¡b¿ Lange.....	langer centuri save australia justin langer hi.....	langer century save australia justin langer hi.....

2) Deep Neural Network: The TF-IDF Deep Neural Network was implemented in PyTorch [4]. It uses dense TF-IDF vectors as input and passes them through multiple fully connected layers:

$$h_l = \mathrm{ReLU}(W_l h_{l-1} + b_l), \tag{6}$$

with dropout after selected hidden layers. The architecture is:

$$\text{Input} \to H \to 128 \to 64 \to 32 \to K, \tag{7}$$

where H is either 256 or 384 hidden units and $K = 4$ is the number of classes. Cross-entropy loss was minimised using Adam:

$$\mathcal{L} = -\sum_{k=1}^{K} y_k \log(\hat{y}_k). \tag{8}$$

3) Recurrent Neural Network: The SimpleRNN treats the headline as a sequence of token indices, maps tokens to embeddings, and updates a hidden state across time:

$$h_t = \phi(W_x x_t + W_h h_{t-1} + b), \tag{9}$$

where ϕ is a non-linear activation. The final hidden state is passed to a classifier head.

4) GRU and LSTM: The GRU adds gates that control how much old information is retained and how much new information is written. The main GRU update is:

$$z_t = \sigma(W_z x_t + U_z h_{t-1}), \tag{10}$$

$$r_t = \sigma(W_r x_t + U_r h_{t-1}), \tag{11}$$

$$\tilde{h}_t = \tanh(W_h x_t + U_h(r_t \odot h_{t-1})), \tag{12}$$

$$h_t = (1 - z_t) \odot h_{t-1} + z_t \odot \tilde{h}_t. \tag{13}$$

The LSTM uses input, forget, and output gates to maintain a memory cell, making it more capable of modelling longer dependencies [6]. Bidirectional variants process the sequence in forward and backward directions and concatenate the final states:

$$h_{final} = [\overrightarrow{h_T}; \overleftarrow{h_1}]. \tag{14}$$

The recurrent classifier head is Dropout, Linear(64), ReLU, and Linear(4). All neural models were trained with Adam [5], batch size 256, and early stopping based on validation accuracy.

III. EXPERIMENTAL SETUP

A. Validation Splits and Run Design

The experiment used three stratified validation split configurations, shown in Table VIII. Stratification preserves class proportions in each train-validation split. The notebook includes 30 separate training runs, and each run changes at least one important factor: preprocessing version, validation split, n-gram range, vocabulary size, regularisation value, hidden units, dropout, embedding dimension, maximum sequence length, recurrent architecture, or training epochs.

TABLE VIII
VALIDATION SPLIT CONFIGURATIONS

Split Name	Validation Size	Seed
split_80_20_seed42	0.20	42
split_75_25_seed7	0.25	7
split_70_30_seed21	0.30	21

B. Evaluation Metrics

The primary metrics are accuracy and macro F1-score. Accuracy is the proportion of correctly classified examples:

$$\text{Accuracy} = \frac{TP + TN}{TP + TN + FP + FN}. \tag{15}$$

For each class, precision, recall, and F1-score are:

$$\text{Precision} = \frac{TP}{TP + FP}, \quad \text{Recall} = \frac{TP}{TP + FN}, \tag{16}$$

$$F1 = 2 \times \frac{\text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}}. \tag{17}$$

Macro F1 is the unweighted mean of F1-scores across all four classes:

$$\text{MacroF1} = \frac{1}{K} \sum_{k=1}^{K} F1_k. \tag{18}$$

Macro F1 is especially suitable here because the training data is imbalanced while the test set is balanced.

IV. RESULTS AND DISCUSSION

A. Validation Results Across 30 Runs

The validation phase ranked all 30 runs by macro F1-score. Table IX shows the top 10 validation configurations from the output log. The best validation experiment was RUN_02: Logistic Regression with TF-IDF, no preprocessing, unigram-bigram features, 30,000 max features, and $C = 2.0$. It achieved 0.923394 validation accuracy and 0.917533 validation macro F1-score. RUN_06, which used Logistic Regression with TF-IDF and optimum preprocessing, was the second-best validation configuration and later became the best final test configuration.

Fig. 7 plots macro F1-score across all 30 validation runs. The graph shows three important patterns. First, TF-IDF Logistic Regression is consistently strong. Second, TF-IDF DNN models are competitive but do not clearly surpass Logistic Regression. Third, recurrent models have higher variance. Some GRU and

TABLE IX
TOP 10 VALIDATION EXPERIMENTS BY MACRO F1-SCORE

Run	Model	Representation	Preprocessing	Val. Acc.	Val. Macro F1	Runtime (s)
RUN_02	Logistic Regression	TF-IDF	no_preprocessing	0.9234	0.9175	6.32
RUN_06	Logistic Regression	TF-IDF	optimum_preprocessing	0.9192	0.9127	4.55
RUN_01	Logistic Regression	TF-IDF	no_preprocessing	0.9185	0.9116	3.09
RUN_07	Deep Neural Network	TF-IDF	no_preprocessing	0.9179	0.9110	6.41
RUN_08	Deep Neural Network	TF-IDF	no_preprocessing	0.9153	0.9093	9.53
RUN_12	Deep Neural Network	TF-IDF	optimum_preprocessing	0.9155	0.9091	7.94
RUN_04	Logistic Regression	TF-IDF	extreme_preprocessing	0.9160	0.9085	4.96
RUN_05	Logistic Regression	TF-IDF	optimum_preprocessing	0.9141	0.9065	2.17
RUN_10	Deep Neural Network	TF-IDF	extreme_preprocessing	0.9138	0.9061	7.63
RUN_29	Bidirectional GRU	Skip-gram Word2Vec	optimum_preprocessing	0.9136	0.9059	12.28

LSTM models perform close to the best TF-IDF models, while some SimpleRNN settings perform poorly. This indicates that recurrent modelling alone is not enough; stable embeddings, adequate sequence length, and suitable preprocessing are also necessary.

B. Final Test Results

After validation, the best configuration for each model-representation-preprocessing group was selected for final test evaluation. The final test set had 12,000 examples and was balanced across the four classes. Table X reports the full final test ranking from the notebook output.

The best final experiment is shown explicitly in Table XI, as requested from the final notebook cell. RUN_06 achieved 0.915833 accuracy and 0.915482 macro F1-score using Logistic Regression, TF-IDF, and optimum preprocessing. The worst final experiment was RUN_19, a SimpleRNN with Skip-gram Word2Vec and extreme preprocessing, which achieved only 0.250167 accuracy and 0.100346 macro F1-score. Since the test set has four balanced classes, an accuracy near 0.25 indicates near single-class prediction behaviour rather than meaningful learning.

Fig. 8 compares final test accuracy across the selected experiments. Fig. 9 compares final test macro F1-score. Both plots support the same conclusion: the top models are concentrated around TF-IDF Logistic Regression, TF-IDF DNN, and bidirectional LSTM or GRU models. However, the final winner is the classical TF-IDF Logistic Regression configuration. The result is important because it shows that a simpler model can outperform more complex neural architectures when the text is short and strongly lexical.

C. Confusion Matrix Analysis

The confusion matrix for the best model is shown in Fig. 10. The best model performs strongly across all four classes. Sports is the easiest class, with very high recall and precision, likely because it has distinctive words such as “game”, “team”, “season”, “win”, and named sports entities. Business and Science and Technology are more likely to be confused because both may contain company names, product names, market terms, and technology-related business events. World News also performs well but has some overlap with Business because international political and economic headlines can share vocabulary.

The classification report for the best model shows Business precision of 0.92, recall of 0.84, and F1-score of 0.88. Science and Technology achieved 0.86 precision, 0.94 recall, and 0.90 F1-score. Sports achieved 0.96 precision, 0.98 recall, and 0.97 F1-score. World News achieved 0.94 precision, 0.90 recall, and 0.92 F1-score. These class-level results explain why the macro F1-score is close to the overall accuracy.

The confusion matrix for the worst model is shown in Fig. 11. RUN_19 predicts almost all samples as Science and Technology. The output classification report confirms that Business, Sports, and World News receive zero or near-zero recall. This explains the macro F1-score of 0.100346. The result suggests that the SimpleRNN architecture with extreme preprocessing and Skip-gram embeddings failed to learn a stable separation boundary. Extreme preprocessing likely removed too many useful surface features, while the SimpleRNN had limited ability to recover the lost information from short sequences.

D. Effect of Preprocessing

The preprocessing comparison is one of the central findings of the project. Extreme preprocessing was designed to apply all common cleaning methods, including stemming and full stopword removal. However, its performance was not consistently better. In some cases, it performed well, such as Logistic Regression RUN_04 with 0.914508 macro F1-score. In other cases, especially SimpleRNN RUN_19, it produced very poor performance. This indicates that aggressive reduction can be risky for short text because each word carries high information value.

Optimum preprocessing provided a better balance. It removed HTML artifacts and common noise while preserving negation and readable lemmas. It produced the best final test result with Logistic Regression and remained competitive for neural models. This supports the idea that preprocessing should not be applied blindly. The best pipeline depends on the task, data length, label semantics, and chosen representation.

The no preprocessing condition also performed surprisingly well in some validation settings. RUN_02 achieved the highest validation macro F1-score, and TF-IDF DNN with no preprocessing achieved 0.906373 final macro F1-score. This suggests that TF-IDF can tolerate some raw formatting noise,

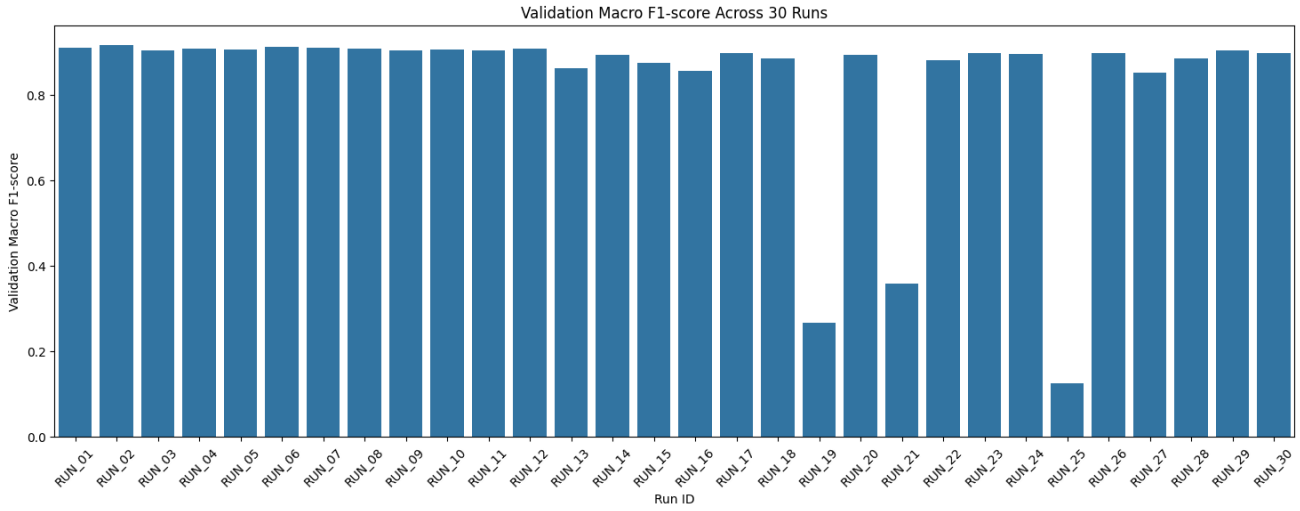

Fig. 7. Validation macro F1-score across 30 training runs. The best validation scores are concentrated among TF-IDF Logistic Regression and TF-IDF DNN models, while sequence models show greater variance.

TABLE X
FINAL TEST RESULTS FOR SELECTED VALIDATION CONFIGURATIONS

Source Run	Model	Representation	Preprocessing	Test Acc.	Test Macro F1	Runtime (s)
RUN_06	Logistic Regression	TF-IDF	optimum_preprocessing	0.9158	0.9155	5.85
RUN_04	Logistic Regression	TF-IDF	extreme_preprocessing	0.9148	0.9145	5.54
RUN_24	Bidirectional LSTM	Skip-gram Word2Vec	extreme_preprocessing	0.9070	0.9064	14.66
RUN_07	Deep Neural Network	TF-IDF	no_preprocessing	0.9065	0.9064	8.11
RUN_30	Bidirectional LSTM	Skip-gram Word2Vec	optimum_preprocessing	0.9061	0.9059	14.85
RUN_20	GRU	Skip-gram Word2Vec	extreme_preprocessing	0.9047	0.9042	11.02
RUN_29	Bidirectional GRU	Skip-gram Word2Vec	optimum_preprocessing	0.9047	0.9041	14.70
RUN_23	Bidirectional GRU	Skip-gram Word2Vec	extreme_preprocessing	0.9019	0.9017	14.78
RUN_12	Deep Neural Network	TF-IDF	optimum_preprocessing	0.9018	0.9016	22.92
RUN_10	Deep Neural Network	TF-IDF	extreme_preprocessing	0.9017	0.9014	16.45
RUN_26	GRU	Skip-gram Word2Vec	optimum_preprocessing	0.9004	0.9001	11.13
RUN_17	Bidirectional GRU	Skip-gram Word2Vec	no_preprocessing	0.9000	0.8996	21.99
RUN_14	GRU	Skip-gram Word2Vec	no_preprocessing	0.8985	0.8984	16.92
RUN_21	LSTM	Skip-gram Word2Vec	extreme_preprocessing	0.8984	0.8984	14.37
RUN_02	Logistic Regression	TF-IDF	no_preprocessing	0.8937	0.8941	8.13
RUN_15	LSTM	Skip-gram Word2Vec	no_preprocessing	0.8939	0.8937	21.56
RUN_22	Bidirectional SimpleRNN	Skip-gram Word2Vec	extreme_preprocessing	0.8938	0.8934	11.32
RUN_18	Bidirectional LSTM	Skip-gram Word2Vec	no_preprocessing	0.8922	0.8912	22.04
RUN_28	Bidirectional SimpleRNN	Skip-gram Word2Vec	optimum_preprocessing	0.8880	0.8878	11.49
RUN_27	LSTM	Skip-gram Word2Vec	optimum_preprocessing	0.8773	0.8763	15.01
RUN_13	SimpleRNN	Skip-gram Word2Vec	no_preprocessing	0.8602	0.8605	17.33
RUN_16	Bidirectional SimpleRNN	Skip-gram Word2Vec	no_preprocessing	0.4950	0.3825	17.36
RUN_25	SimpleRNN	Skip-gram Word2Vec	optimum_preprocessing	0.2503	0.1007	11.59
RUN_19	SimpleRNN	Skip-gram Word2Vec	extreme_preprocessing	0.2502	0.1003	11.26

TABLE XI
BEST AND WORST FINAL TEST EXPERIMENTS FROM THE LAST NOTEBOOK OUTPUT

Case	Run	Model	Representation	Preprocessing	Accuracy	Macro F1	Runtime (s)
Best	RUN_06	Logistic Regression	TF-IDF	optimum_preprocessing	0.915833	0.915482	5.85
Worst	RUN_19	SimpleRNN	Skip-gram Word2Vec	extreme_preprocessing	0.250167	0.100346	11.26

especially when the training and validation data share similar formatting patterns. However, the final test result favoured optimum preprocessing, which generalised better than the raw version.

E. Effect of Representation

The results show a strong advantage for TF-IDF in this dataset. TF-IDF directly represents exact words and n-grams, which are highly informative for news topics. For example, “red sox”, “world cup”, “oil prices”, “microsoft”, and “prime

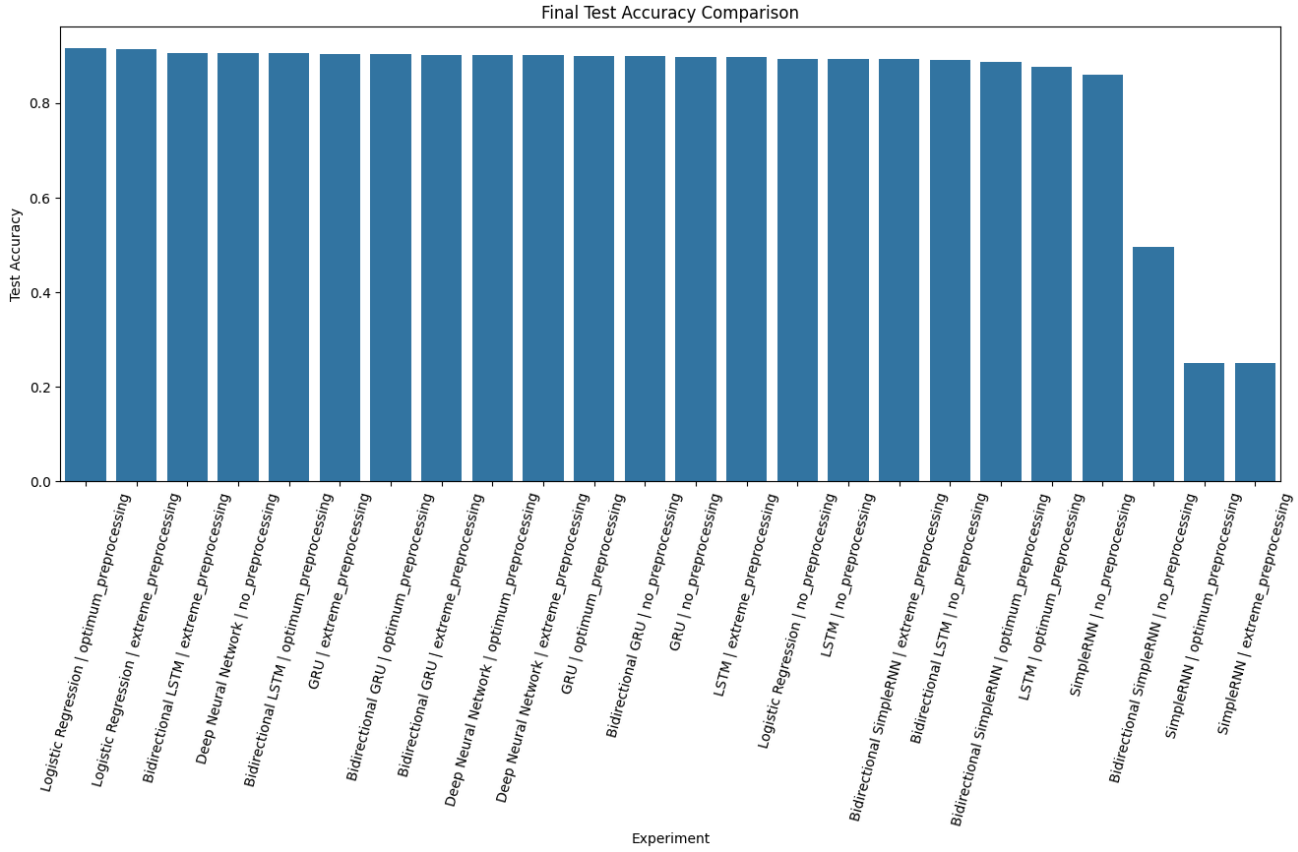

Fig. 8. Final test accuracy comparison for all selected configurations.

minister" can directly support class separation. Because headlines are short and topic cues are lexical, sparse features are well suited to the task.

Skip-gram Word2Vec provides dense semantic representations, but it did not outperform TF-IDF overall. One reason is that Word2Vec embeddings were trained only on the project training data, not on a very large external corpus. Another reason is that dense embeddings can smooth over exact lexical distinctions that are valuable for news category detection. The recurrent models based on Word2Vec performed respectably when using GRU or LSTM variants, but their best scores still remained below the best TF-IDF Logistic Regression model.

F. Effect of Model Architecture

Logistic Regression was the most reliable model in the study. It is linear, fast, and interpretable, but its high-dimensional TF-IDF input gives it enough expressive power to separate the four topic classes. The best model also had low runtime, taking 5.85 seconds for final test evaluation.

The TF-IDF DNN models were competitive but did not outperform Logistic Regression. This is not unexpected. Dense neural networks can model nonlinear interactions, but converting sparse TF-IDF into dense arrays increases computation and may not add much value when the class boundaries are already well captured by linear lexical features.

Among recurrent models, GRU and LSTM were more stable than SimpleRNN. Bidirectional LSTM with extreme preprocessing reached 0.906426 final macro F1-score, and bidirectional LSTM with optimum preprocessing reached 0.905916. These are strong results, but they required more training time and did not beat the simpler best baseline. SimpleRNN was unstable and produced the worst final result, which is consistent with the known limitation of vanilla RNNs in modelling longer dependencies and avoiding gradient-related issues.

V. Conclusion

This project developed and evaluated a full NLP pipeline for multi-class classification of news headlines. The work included data inspection, EDA, duplicate removal, three preprocessing pipelines, TF-IDF and Skip-gram Word2Vec representations, 30 validation experiments, selected final test evaluations, confusion matrices, and comparative analysis.

The best final model was Logistic Regression with TF-IDF and optimum preprocessing, achieving 0.915833 accuracy and 0.915482 macro F1-score on the balanced 12,000-sample test set. This result shows that classical machine learning remains highly effective for short text classification when paired with a strong feature representation and thoughtful preprocessing. The worst model, SimpleRNN with Skip-gram Word2Vec

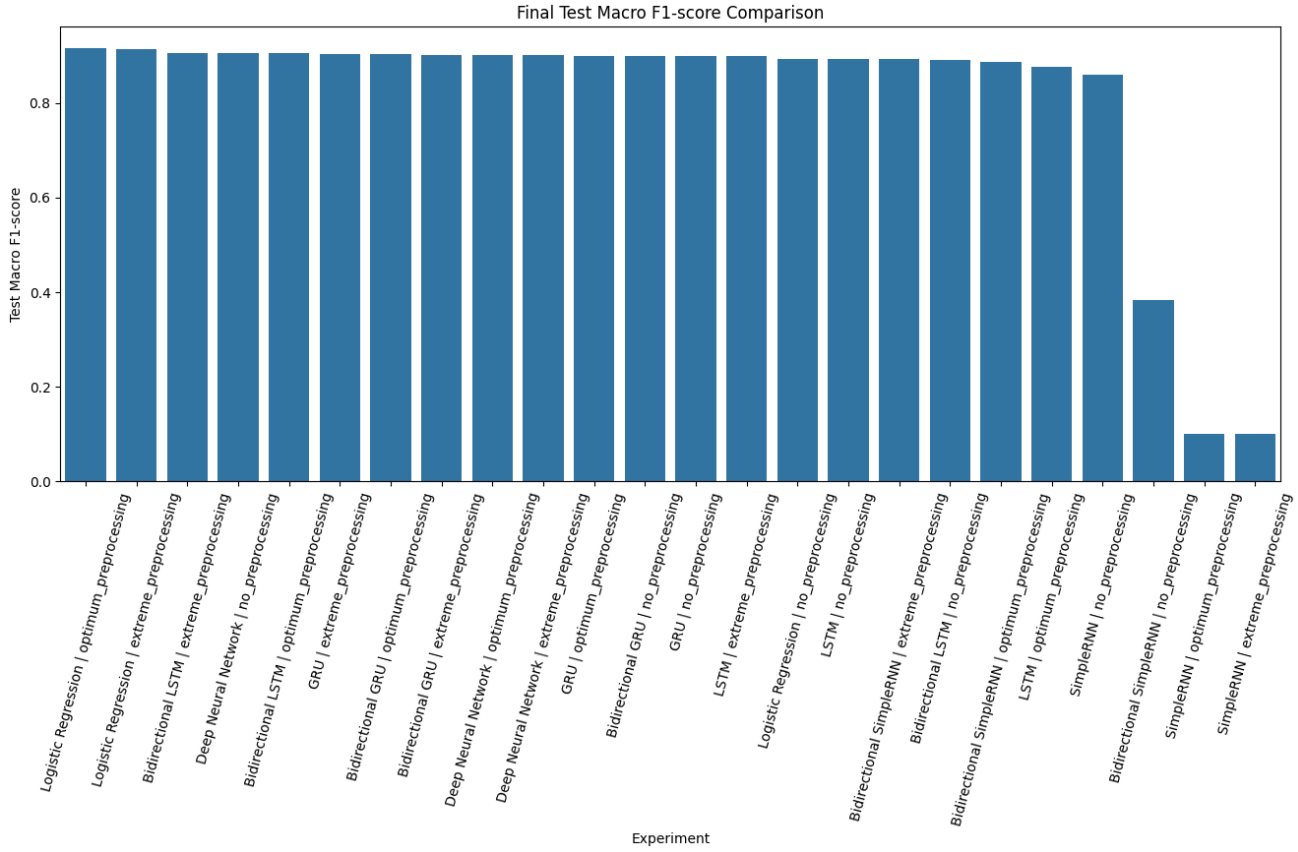

Fig. 9. Final test macro F1-score comparison for all selected configurations.

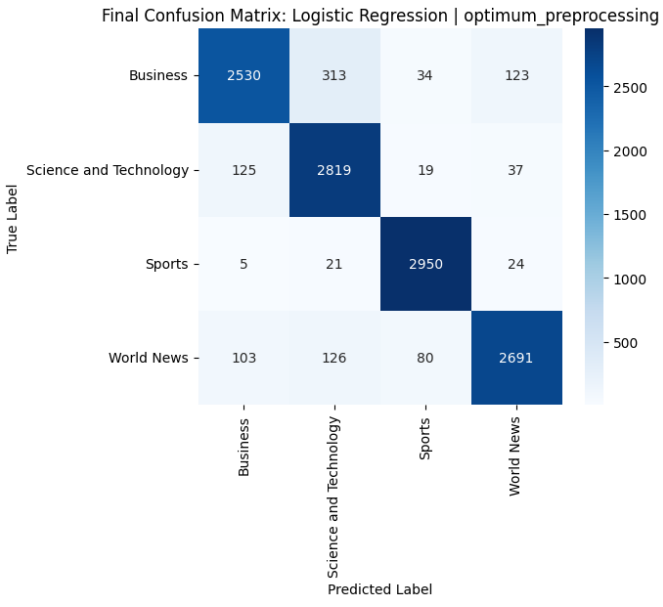

Fig. 10. Final confusion matrix for the best experiment, RUN_06: Logistic Regression with TF-IDF and optimum preprocessing.

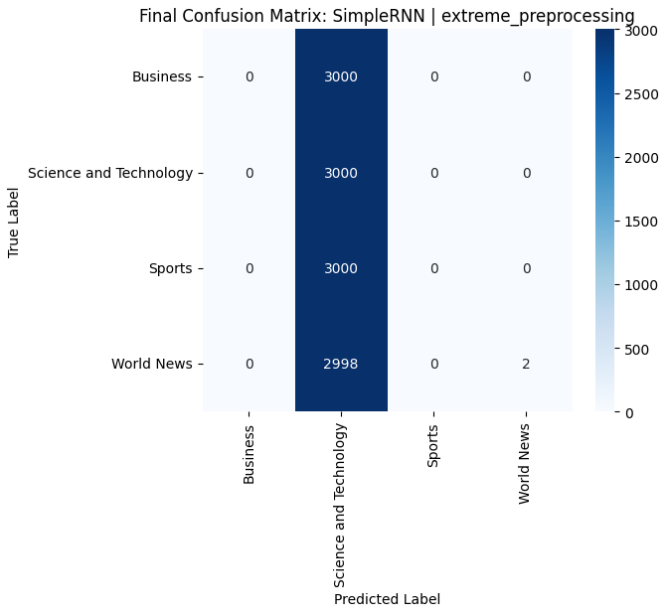

Fig. 11. Final confusion matrix for the worst experiment, RUN_19: SimpleRNN with Skip-gram Word2Vec and extreme preprocessing.

and extreme preprocessing, achieved only 0.250167 accuracy and 0.100346 macro F1-score, showing that neural sequence models can fail when preprocessing is too aggressive and the

architecture is not expressive enough.

The most important takeaway is that preprocessing must be data-driven. Extreme preprocessing is not automatically better. For this dataset, optimum preprocessing worked best because it cleaned HTML and noise while preserving useful lexical forms and negation. TF-IDF performed especially well because the labels are strongly connected to visible words and phrases. Neural recurrent models performed well in several configurations, especially GRU and LSTM, but their additional complexity did not translate into the best final score.

The main limitations are that the experiments used one assigned dataset, trained Word2Vec only on the project corpus, and did not include transformer-based contextual embeddings such as BERT. Future work could evaluate pre-trained transformer models, class-weighted training for the imbalanced training set, external pre-trained embeddings, calibration analysis, and explainability methods that identify the most influential words per class. Another useful extension would be ablation testing on the preprocessing pipeline to isolate the individual effect of stopword removal, lemmatisation, stemming, and n-gram features.

REFERENCES

[1] G. Salton and C. Buckley, "Term-weighting approaches in automatic text retrieval," *Information Processing and Management*, vol. 24, no. 5, pp. 513–523, 1988.

[2] T. Mikolov, K. Chen, G. Corrado, and J. Dean, "Efficient estimation of word representations in vector space," *arXiv preprint arXiv:1301.3781*, 2013.

[3] F. Pedregosa *et al.*, "Scikit-learn: Machine learning in Python," *Journal of Machine Learning Research*, vol. 12, pp. 2825–2830, 2011.

[4] A. Paszke *et al.*, "PyTorch: An imperative style, high-performance deep learning library," in *Advances in Neural Information Processing Systems*, 2019, pp. 8024–8035.

[5] D. P. Kingma and J. Ba, "Adam: A method for stochastic optimization," in *International Conference on Learning Representations*, 2015.

[6] S. Hochreiter and J. Schmidhuber, "Long short-term memory," *Neural Computation*, vol. 9, no. 8, pp. 1735–1780, 1997.

[7] K. Cho *et al.*, "Learning phrase representations using RNN encoder-decoder for statistical machine translation," in *Proc. Conf. Empirical Methods in Natural Language Processing*, 2014, pp. 1724–1734.

[8] G. A. Miller, "WordNet: A lexical database for English," *Communications of the ACM*, vol. 38, no. 11, pp. 39–41, 1995.

[9] M. F. Porter, "An algorithm for suffix stripping," *Program*, vol. 14, no. 3, pp. 130–137, 1980.

[10] W. McKinney, "Data structures for statistical computing in Python," in *Proc. 9th Python in Science Conference*, 2010, pp. 56–61.

[11] J. D. Hunter, "Matplotlib: A 2D graphics environment," *Computing in Science and Engineering*, vol. 9, no. 3, pp. 90–95, 2007.

[12] S. Bird, E. Klein, and E. Loper, *Natural Language Processing with Python*. Sebastopol, CA, USA: O'Reilly Media, 2009.